

# 1. Executive Summary

SECURITY SCORE

10

CRITICAL

0

HIGH

12

MEDIUM

15

INFO/LOW

8

## 1.2 ISMS-P 컴플라이언스 위반 요약

이번 분석에서 총 41건의 ISMS-P 인증기준 위반 또는 위반 의심 사례가 탐지되었습니다. 관련 부서는 본 리포트의 세부 항목을 검토 후 조치 요망.

| # | ISMS-P 위반 핵심 내용                                     | 관련 취약점                          | 심각도    |
|---|-----------------------------------------------------|---------------------------------|--------|
| 1 | ISMS-P 2.10.2 클라우드 보안: 클라우드 서비스 이용 시 비인가 접근 및 설...  | 쿠버네티스 컨테이너 권한 상승 방지 설정 미흡       | MEDIUM |
| 2 | ISMS-P 2.6 접근통제 및 안전 조치 취지 위반 가능성. 시스템/서비스 실행 계정... | Docker 컨테이너가 root 사용자로 실행될 수 있음 | HIGH   |
| 3 | ISMS-P 2.11.2 취약점 점검 및 조치                           | 로그인 폼 조회용 엔드포인트의 HTTP 메서드 미지정   | MEDIUM |
| 4 | 해당 문자열이 실제 인증정보로 사용된다면 개인정보/인증정보의 보호 측면에서 안전조치 ...  | 하드코딩된 샘플 JWT 문자열 탐지             | LOW    |

|    |                                                     |                                                                   |        |
|----|-----------------------------------------------------|-------------------------------------------------------------------|--------|
| 5  | 개인정보 및 중요정보 처리 시스템의 안전조치 의무, 접근통제, 최소권한 원칙 미준수 ...  | Docker 컨테이너가 non-root 사용자 없이 실행될 수 있음                             | HIGH   |
| 6  | 접근권한 관리 및 시스템 권한 최소화 요구사항 위반 가능성                    | 컨테이너 실행 시 최소권한 원칙이 적용되지 않음                                        | HIGH   |
| 7  | ISMS-P 2.10.1 사고 예방 및 대응체계 구축 관점에서, 침해사고 발생 시 권...  | Kubernetes 컨테이너에 privilege escalation 방지 설정 누락                    | MEDIUM |
| 8  | ISMS-P 관점에서 시스템 접근 통제 및 최소권한 원칙 미흡에 해당할 수 있습니다. ... | Kubernetes 컨테이너 권한 상승 차단 설정 미흡                                    | MEDIUM |
| 9  | 2.10.8 패치관리, 2.8.6 운영환경 이관, 2.9.2 성능 및 장애관리 (보안 ... | Kubernetes 컨테이너가 root 사용자로 실행될 가능성                                | INFO   |
| 10 | ISMS-P 2.10.8 패치관리 및 안전조치의무 관점에서, 기본 실행 권한을 최소화...  | Kubernetes 컨테이너 비루트 실행 설정 누락 가능성                                  | INFO   |
| 11 | ISMS-P 2.10.8 패치관리 및 안전조치 의무의 취지에 따라, 최신 보안 설정/...  | Kubernetes 컨테이너의 비루트 실행 보안설정 미흡                                   | INFO   |
| 12 | ISMS-P 2.10.2 클라우드 보안: 클라우드 서비스 이용 시 설정 오류로 인한 ...  | Kubernetes 컨테이너에 권한 상승 방지 설정 누락                                   | MEDIUM |
| 13 | 2.11.2 취약점 점검 및 조치, 2.9.3 공개서버 보안                   | Kubernetes 컨테이너가 비루트(non-root)로 실행되도록 보안 설정이 필요함                  | INFO   |
| 14 | ISMS-P 2.10.8 패치관리 및 안전한 시스템 운영 취지에 반하며, 컨테이너 보...  | Nginx 컨테이너에 privilege escalation 방지용 securityContext가 없어 권한 상승 가능 | MEDIUM |
| 15 | ISMS-P 2.9.1 변경관리, 2.10.8 패치관리, 1.1.5 정책 수립         | Kubernetes StatefulSet의 PostgreSQL 컨테이너가 비루트 실행을 보장하지 않음          | INFO   |
| 16 | ISMS-P 2.9.1 변경관리, 2.10.8 패치관리                      | 컨테이너 실행 권한 최소화 설정 미흡                                              | INFO   |
| 17 | 2.10.2 클라우드 보안 - 클라우드 서비스 이용 시 설정 오류로 중요정보/개인정보...  | Kubernetes 컨테이너에 권한 상승 방지 설정 누락                                   | MEDIUM |
| 18 | 2.10.2 클라우드 보안                                      | Kubernetes DB 워크로드의 기본 보안 설정 미흡                                   | MEDIUM |

|    |                                                     |                                                      |        |
|----|-----------------------------------------------------|------------------------------------------------------|--------|
| 19 | ISMS-P의 접근통제 및 시스템 보안 설정 미흡에 해당할 수 있습니다. 특히 서비스...  | Kubernetes 컨테이너 권한 상승 방지 설정 미흡                       | MEDIUM |
| 20 | 2.8.6 운영환경 이관: 운영환경에는 서비스 실행에 필요한 안전한 설정만 배포되어야...  | Kubernetes 컨테이너 비루트 실행 설정 누락                         | INFO   |
| 21 | ISMS-P 2.6.1 접근통제 및 안전 조치 취지 위반 가능. 컨테이너 권한 통제 미... | Kubernetes 컨테이너에 privilege escalation 차단 설정 누락       | MEDIUM |
| 22 | 2.6.1 네트워크 접근: 컨테이너 보안 설정 미흡으로 침해 시 내부 네트워크 및 민...  | Kubernetes Pod/Container가 비루트 사용자 실행 강제 설정 없이 배포됨    | INFO   |
| 23 | 2.9.4 업무용 단말기기 보안                                   | Kubernetes 컨테이너가 비루트 실행 보장이 없어 권한 상승 위험이 있음          | INFO   |
| 24 | 2.9.4 업무용 단말기기 보안                                   | initContainer/컨테이너 비루트 실행 설정 미흡                      | INFO   |
| 25 | ISMS-P 2.10.2 클라우드 보안, 2.6.1 네트워크 접근통제 관련 보호대책 ...  | Kubernetes nginx 컨테이너에 privilege escalation 차단 설정 누락 | MEDIUM |
| 26 | ISMS-P 3. 개인정보 보호 조직/ 접근 권한 관리, 6. 접근 통제에 관한 사항 ... | 클라이언트가 userId를 직접 지정하는 작품 등록 API로 인한 IDOR 가능성        | HIGH   |
| 27 | 개인정보 보호법 제29조 안전 조치의무, ISMS-P 2.6 접근통제 위반 가능성       | 비밀번호 변경 API에 클라이언트 저장 userId를 그대로 사용하여 계정 바인딩 우회 가능  | HIGH   |
| 28 | ISMS-P 1.1.6, 2.8.6                                 | 비밀번호 찾기 결과로 임시 비밀번호를 직접 노출하는 계정 복구 로직                | HIGH   |
| 29 | 2.5.5 특수 계정 및 권한 관리 / 1.2.1 정보자산 식별                 | 로그인 성공 후 사용자 식별 정보를 클라이언트 localStorage에 저장           | MEDIUM |
| 30 | 2.9.1 변경관리                                          | 로그인 성공 조건을 응답 본문의 status 값에 의존                       | MEDIUM |
| 31 | 2.5.5 특수 계정 및 권한 관리                                 | API 키 검증과 사용자 인증이 없는 이미지 생성 프록시 엔드포인트                | HIGH   |
| 32 | 176 이상행위 분석 및 모니터링                                  | 요청량 제한 부재로 인한 비용 고갈 및 이상 트래픽 통제 미흡                   | MEDIUM |
| 33 | 2.5.5 특수 계정 및 권한 관리                                 | 클라이언트 제어 model 값에 대한 허용 목록 검증 부재                     | MEDIUM |

|    |                                                     |                                                        |        |
|----|-----------------------------------------------------|--------------------------------------------------------|--------|
| 34 | 개인정보보호법 제29조(안전 조치의무) 및 ISMS-P 접근통제/권한관리 취지 위반 가... | 클라이언트가 userId와 리소스 ID를 임의 지정하는 IDOR 가능성                | HIGH   |
| 35 | 정보주체의 계정 정보에 대한 접근통제 미흡, 권한 최소화 및 계정별 접근통제 정책 위...  | 비밀번호 변경 API에서 세션 사용자 검증 없이 요청 userId를 신뢰하는 권한 우회(IDOR) | HIGH   |
| 36 | 개인정보 및 중요정보에 대한 부적절한 인증 우회, 열람·정정·삭제 등 권리보장 절차 ...  | 본인 확인 절차 없이 임시 비밀번호를 발급하는 비밀번호 찾기 흐름                   | HIGH   |
| 37 | 개인정보 처리 단계별 보호조치 미흡, 접근권한 오·남용 방지 통제 미흡             | 비밀번호 변경 요청에 클라이언트 지정 userId 포함으로 인한 IDOR 가능성 검토 필요     | INFO   |
| 38 | 접근권한 검토 및 권한 오·남용 통제 미흡                             | 현재 비밀번호 검증 수행 여부는 상위 로직 확인 필요                          | INFO   |
| 39 | 2.5.5 특수 계정 및 권한 관리, 2.9.1 변경관리, 개인정보의 안전성 확보조치...  | 평문 비밀번호를 이메일과 함께 직접 조회하는 인증 로직 결함                      | HIGH   |
| 40 | 2.6.1 접근통제: 인가되지 않은 계정 식별자(userId)를 이용한 자원 접근/변...  | 사용자 비밀번호 변경 API의 권한 검증 부재로 인한 IDOR                     | HIGH   |
| 41 | 2.8.6 운영환경 이관: 운영계에서 동시성/정합성 문제를 유발할 수 있는 변경을 통...  | 회원가입 이메일 중복 확인과 저장 사이의 경쟁조건                            | MEDIUM |

## 2. Detailed Findings

### 2.1 코드 정적 분석 결과

#### 1. [HIGH] Docker 컨테이너가 root 사용자로 실행될 수 있음

HIGH | TP | dockerfile.security.missing-user-entrypoint.missing-user-entrypoint | mini\_project5\Dockfile

최종 런타임 stage의 Dockerfile에 USER 지시어가 없고, ENTRYPOINT로 실행되는 Java 프로세스가 기본적으로 root 권한으로 동작할 수 있습니다. 컨테이너 내부에서 애플리케이션이 침해되거나 RCE가 발생할 경우 root 권한으로 파일 시스템 변경, 추가 도구 설치, 설정 변조, 권한 상승 시도 등의 영향이 커집니다. 따라서 최종 stage에서 비루트 전용 사용자로 실행하도록 명시해야 합니다.

공격 시나리오: 공격자가 별도의 애플리케이션 취약점을 통해 컨테이너 내부에서 명령 실행 권한을 얻으면, 해당 프로세스는 root로 동작하므로 /app 이하뿐 아니라 컨테이너 파일 시스템 전체를 수정하거나 민감 파일을 열람할 수 있습니다. 이미지에 불필요한 시스템 도구가 존재하거나 호스트 마운트가 연결된 경우 피해 범위가 더 커질 수 있습니다.

취약 코드:

```
7 FROM amazoncorretto:17-alpine-jre
8 WORKDIR /app
9 COPY --from=builder /app/build/libs/*.jar app.jar
10 EXPOSE 8080
11 ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM amazoncorretto:17-alpine-jre
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=builder /app/build/libs/*.jar app.jar
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 최종 런타임 stage에 비루트 사용자(appuser)를 생성하고 USER 지시어를 추가하여 애플리케이션이 root로 실행되지 않도록 수정합니다. 필요 시 파일 소유권도 함께 조정해 런타임 시 권한 오류가 발생하지 않도록 합니다. Kubernetes 배포 시에는 securityContext(runAsNonRoot, runAsUser)도 추가하여 이중 방어를 권장합니다.

**ISMS-P:** ISMS-P 2.6 접근통제 및 안전조치 취지 위반 가능성. 시스템/서비스 실행 계정이 과도한 권한(root)으로 동작하면 비인가 접근 시 영향도가 확대되며, 최소권한 원칙과 접근

통제 강화 요구에 부합하지 않습니다.

## 2. [HIGH] Docker 컨테이너가 non-root 사용자 없이 실행될 수 있음

HIGH | TP | dockerfile.security.missing-user-entrypoint.missing-user-entrypoint | mini\_project5\book\Dockerfile

Dockerfile에 USER 지시문이 없고 ENTRYPOINT로 애플리케이션을 바로 실행하고 있어, 컨테이너 내부 프로세스가 기본적으로 root 권한으로 동작할 가능성이 있습니다. 이는 컨테이너 탈취, 파일 시스템 조작, 권한 상승 및 피해 범위 확대를 유발할 수 있는 보안 구성 결함입니다.

공격 시나리오: 애플리케이션 내부에 RCE, 파일 업로드 경로 조작, 템플릿/명령 실행 취약점이 존재할 경우 공격자는 root 컨테이너 권한으로 파일 수정, 민감정보 열람, 추가 악성 파일 배치, 런타임 환경 변조를 시도할 수 있습니다. K8s 환경에서는 잘못된 볼륨/권한 설정과 결합 시 영향이 더 커질 수 있습니다.

취약 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk

WORKDIR /app
COPY build/libs/*.jar app.jar

EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
FROM public.ecr.aws/amazoncorretto/amazoncorretto:17-al2-jdk

WORKDIR /app
COPY build/libs/*.jar app.jar

RUN adduser --disabled-password --gecos "" appuser \
    && chown -R appuser:appuser /app

USER appuser
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: 컨테이너가 root로 실행되지 않도록 non-root 계정을 생성한 뒤 USER를 명시해야 합니다. 또한 애플리케이션 파일과 WORKDIR의 소유권을 해당 사용자에게 부여해 실행 시 권한 문제를 방지해야 합니다. 가능하다면 K8s 보안 컨텍스트 (runAsNonRoot, allowPrivilegeEscalation=false 등)도 함께 적용하세요.

**ISMS-P:** 개인정보 및 중요정보 처리 시스템의 안전조치 의무, 접근통제, 최소권한 원칙 미준수 가능성

### 3. [HIGH] 컨테이너 실행 시 최소권한 원칙이 적용되지 않음

HIGH | TP | `dockerfile.security.missing-user-entriypoint.missing-user-entriypoint` | `mini_project5\book\Dockefile`

Dockerfile이 최종 실행 사용자(USER)를 지정하지 않아 애플리케이션이 root 권한으로 시작될 수 있습니다. 이는 보안상 바람직하지 않으며, 컨테이너 내부 침해 시 피해를 크게 확대시킬 수 있는 구성 취약점입니다.

공격 시나리오: 컨테이너 내에서 취약한 라이브러리 또는 애플리케이션 버그가 발생하면, root 권한으로 실행 중인 프로세스를 통해 시스템 파일 접근/변조, 설정 탈취, 권한 있는 포트/자원 조작이 가능해질 수 있습니다.

취약 코드:

```
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 코드:

```
USER appuser
ENTRYPOINT ["java", "-jar", "app.jar"]
```

수정 가이드: ENTRYPOINT 전에 non-root 사용자를 명시하고, 해당 사용자가 애플리케이션을 실행할 수 있도록 파일 권한을 사전에 조정합니다. 베이스 이미지에 따라 사용자 생성 명령은 다를 수 있으므로 distroless 또는 전용 런타임 이미지를 사용하는 것도 좋은 대안입니다.

**ISMS-P:** 접근권한 관리 및 시스템 권한 최소화 요구사항 위반 가능성

### 4. [MEDIUM] 쿠버네티스 컨테이너 권한 상승 방지 설정 미흡

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | `mini_project5\cluster-autoscaler-autodiscover.yaml`

해당 Deployment의 컨테이너에 `securityContext.allowPrivilegeEscalation: false`가 명시되지 않아, 컨테이너 내부의 `setuid/setgid` 바이너리 또는 권한 상승 가능한 프로세스가 실행될 경우 불필요한 권한 상승을 허용할 수 있습니다. 파드 수준의 `runAsNonRoot/runAsUser` 설정만으로는 `privilege escalation` 차단이 충분하지 않으며, 컨테이너 단위의 보안 설정이 필요합니다.

공격 시나리오: 공격자가 컨테이너 내부의 다른 취약점(예: RCE, 공급망 오염, 취약한 바이너리 실행)을 먼저 확보한 뒤, `setuid/setgid` 파일이나 권한 상승 가능한 실행 흐름을 이용해 컨테이너 내 root 권한 또는 더 높은 권한으로 전환할 수 있습니다. 이후 클러스터 오토스케일러가 사용하는 AWS 자격 증명/서비스 계정 권한, 마운트된 자원, 네임스페이스

권한을 악용해 클러스터 내 자원 조작 또는 민감정보 접근으로 확장될 위험이 있습니다.

취약 코드:

```
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        fsGroup: 65534
      containers:
        - image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.32.1
          name: cluster-autoscaler
```

수정 코드:

```
spec:
  template:
    spec:
      securityContext:
        runAsNonRoot: true
        runAsUser: 65534
        fsGroup: 65534
      containers:
        - name: cluster-autoscaler
          image: registry.k8s.io/autoscaling/cluster-autoscaler:v1.32.1
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            capabilities:
              drop: ["ALL"]
```

수정 가이드: 컨테이너 단위 securityContext에 allowPrivilegeEscalation: false를 명시하여 setuid/setgid 기반 권한 상승을 차단해야 합니다. 가능하다면 readOnlyRootFilesystem: true와 capabilities.drop: ["ALL"]도 함께 적용해 공격 표면을 더 줄이세요. 파드 수준 설정만으로 충분하다고 가정하지 말고, 각 컨테이너에 보안 정책을 직접 적용하는 방식으로 수정하는 것이 안전합니다.

**ISMS-P:** ISMS-P 2.10.2 클라우드 보안: 클라우드 서비스 이용 시 비인가 접근 및 설정 오류로 중요정보가 유·노출되지 않도록 보안 구성과 설정 기준을 수립·이행해야 하며, 관리자 권한은 최소화하고 강화된 접근통제를 적용해야 함. 또한 운영 환경의 보안 설정 적절성을 정기적으로 검토해야 함.

## 5. [MEDIUM] 로그인 폼 조회용 엔드포인트의 HTTP 메서드 미지정

MEDIUM | FP | java.spring.security.unrestricted-request-mapping.unrestricted-request-mapping |

mini\_project5\book\src\main\java\com\aiivle0102\book\controller\UserControlle

@RequestMapping에 method 속성이 없어 모든 HTTP 메서드가 매핑될 수 있으나, 본 메서드는 단순히 로그인 폼을 반환하거나 로그인 세션 여부에 따라 리다이렉트하는 화면 처리용 로직이다. 상태 변경이 없어 CSRF로 인한 직접적인 영향은 성립하지 않는다.

오답 사유: 해당 엔드포인트는 로그인 폼 화면을 반환하거나 이미 로그인된 경우 메인으로 리다이렉트하는 조회성 로직만 수행하며, 세션 값 변경/DB 변경/권한 변경 등 상태 변경이 없다. 따라서 @RequestMapping에 HTTP method가 명시되지 않았다는 사실만으로 실제 CSRF 피해가 발생한다고 보기 어렵다. 스니펫 기준으로는 공격자가 의미 있는 상태 변경 요청을 유발할 수 없으므로 오답으로 판단한다.

공격 시나리오: 공격자가 /login/form 경로로 GET/POST 등을 보내도 서버는 로그인 폼을 반환하거나 이미 로그인된 사용자를 /main으로 리다이렉트할 뿐이다. 세션/계정/권한/데이터가 변경되지 않으므로 CSRF를 통해 탈취하거나 변조할 수 있는 동작이 없다.

수정 가이드: 이 엔드포인트는 화면 조회 목적이므로 GET으로 명시하여 불필요한 HTTP 메서드 허용을 제거한다. 기능상 상태 변경이 없는지 계속 유지하고, 만약 이후 로그인 처리/세션 변경 로직이 추가되면 POST 전용 매핑과 CSRF 토큰 검증을 함께 적용한다.

**ISMS-P:** ISMS-P 2.11.2 취약점 점검 및 조치

## 6. [MEDIUM] Kubernetes 컨테이너에 **privilege escalation** 방지 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext |`  
`mini_project5\k8s-deployment.yaml`

StatefulSet의 postgres 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation이 기본값(true)으로 동작할 가능성이 있습니다. 이 경우 컨테이너 내부에 setuid/setgid 바이너리 등이 존재할 때 권한 상승 위험이 증가하므로, 명시적으로 allowPrivilegeEscalation: false를 설정하는 것이 필요합니다.

공격 시나리오: 공격자가 애플리케이션의 다른 취약점(RCE, 파일 업로드 악용 등)으로 컨테이너 내부 실행 권한을 얻은 경우, allowPrivilegeEscalation이 허용된 상태에서는 setuid/setgid 프로그램이나 잘못된 파일 권한을 이용해 컨테이너 내 권한을 더 높일 수 있습니다. 이후 DB 자격 증명이나 마운트된 볼륨 데이터를 접근하거나, 동일 노드 상의 다른 워크로드에 영향 범위를 넓힐 수 있습니다.

취약 코드:

```
spec:
  template:
    spec:
```

```
containers:
  - name: postgres
    image: postgres:15-alpine
```

수정 코드:

```
spec:
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            runAsNonRoot: true
            capabilities:
              drop:
                - ALL
```

수정 가이드: postgres 컨테이너에 securityContext를 추가해 권한 상승을 차단하십시오. 최소한 allowPrivilegeEscalation을 false로 설정하고, 가능하면 runAsNonRoot 및 capabilities.drop: ["ALL"]도 함께 적용해 컨테이너 하드닝을 강화합니다. PostgreSQL 데이터 디렉터리는 PVC로 분리되어 있으므로 readOnlyRootFilesystem 적용 가능 여부를 검토한 뒤 함께 적용하는 것이 좋습니다.

**ISMS-P:** ISMS-P 2.10.1 사고 예방 및 대응체계 구축 관점에서, 침해사고 발생 시 권한 상승을 통한 피해 확산을 예방하는 보안통제가 미흡한 상태로 볼 수 있습니다.

## 7. [MEDIUM] Kubernetes 컨테이너 권한 상승 차단 설정 미흡

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

backend 컨테이너에 securityContext가 정의되어 있지 않아 allowPrivilegeEscalation 차단이 적용되지 않았습니다. 이 설정 부재는 이미지 내부의 setuid/setgid 바이너리 또는 잘못된 권한 구성을 악용한 컨테이너 내 권한 상승 가능성을 높이며, 최소 권한 원칙에 위배됩니다.

공격 시나리오: 공격자가 애플리케이션 취약점이나 내부 셸 접근을 통해 backend 컨테이너에 진입한 뒤, 이미지 내 setuid/setgid 바이너리 또는 권한이 과도한 실행 파일을 이용해 추가 권한을 획득할 수 있습니다. 권한 상승이 허용된 상태에서는 컨테이너 내 영향 범위가 확대되어 민감 정보 탈취, 호스트/클러스터 자원 접근 시도, 서비스 변조로 이어질 수 있습니다.

취약 코드:

```
containers:
  - name: backend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-backend:lat
    ports:
      - containerPort: 8080
```

수정 코드:

```
containers:
  - name: backend
    image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-backend:lat
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      capabilities:
        drop: ["ALL"]
    ports:
      - containerPort: 8080
```

수정 가이드: backend 컨테이너에 securityContext를 추가하고 allowPrivilegeEscalation을 false로 설정하여 런타임 권한 상승을 차단해야 합니다. 가능하다면 runAsNonRoot와 capabilities.drop: ["ALL"]도 함께 적용해 최소 권한 원칙을 강화하세요. 이미지가 non-root 실행을 지원하는지 사전 확인이 필요합니다.

**ISMS-P:** ISMS-P 관점에서 시스템 접근통제 및 최소권한 원칙 미흡에 해당할 수 있습니다. 특히 컨테이너/서버 운영 시 불필요한 권한 상승 가능성을 방치하는 것은 보호대책 미흡으로 해석될 수 있습니다.

## 8. [MEDIUM] Kubernetes 컨테이너에 권한 상승 방지 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

frontend Deployment의 컨테이너 정의에 securityContext가 없고, 특히 allowPrivilegeEscalation:false가 명시되지 않아 컨테이너 내부의 setuid/setgid 바이너리 또는 취약한 실행 흐름을 통한 권한 상승 가능성을 줄이지 못합니다. Kubernetes 보안 설정 누락으로 분류되는 구성 취약점입니다.

공격 시나리오: 공격자가 컨테이너 내부에서 애플리케이션 취약점이나 이미지에 포함된 setuid/setgid 바이너리를 악용해 셸 접근을 확보한 뒤, allowPrivilegeEscalation 제한이 없어 상위 권한으로 전환하거나 더 넓은 파일/프로세스 권한을 획득할 수 있습니다.

취약 코드:

```
240: - name: frontend
241:   image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-frontend:lat
```

```

242:   ports:
243:     - containerPort: 3000
244:   resources:
245:     requests:
246:       cpu: "100m"
247:       memory: "256Mi"
248:     limits:
249:       cpu: "500m"
250:       memory: "512Mi"

```

수정 코드:

```

- name: frontend
  image: 416225317325.dkr.ecr.us-east-1.amazonaws.com/ai0102-front
  ports:
    - containerPort: 3000
  resources:
    requests:
      cpu: "100m"
      memory: "256Mi"
    limits:
      cpu: "500m"
      memory: "512Mi"
  securityContext:
    allowPrivilegeEscalation: false
    privileged: false
    runAsNonRoot: true
... (3줄 생략)

```

수정 가이드: 컨테이너별 securityContext를 추가하고 allowPrivilegeEscalation을 false로 설정해 권한 상승을 차단해야 합니다. 가능하면 non-root 실행, 읽기 전용 루트 파일시스템, capabilities drop까지 함께 적용해 컨테이너 탈출 및 로컬 권한 상승 위험을 추가로 낮추는 것이 좋습니다.

**ISMS-P:** ISMS-P 2.10.2 클라우드 보안: 클라우드 서비스 이용 시 설정 오류로 인한 비인가 접근/권한 오남용을 방지하기 위한 보안 구성 및 설정 기준을 수립·이행해야 하나, 컨테이너 보안 설정이 미흡합니다.

## 9. [MEDIUM] Nginx 컨테이너에 **privilege escalation** 방지용 **securityContext**가 없어 권한 상승 가능

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-deployment.yaml`

해당 Kubernetes Deployment의 nginx 컨테이너에는 securityContext.allowPrivilegeEscalation 설정이 존재하지 않습니다. Kubernetes 기본 동작상 명시적으로 false를 설정하지 않으면 컨테이너 내부의 setuid/setgid 바이너리

~~또는 취약한 런타임 동작을 통해 권한 상승 위험이 남습니다. 제공된 스니펫 범위에서는 상위 Pod 수준의 보안 설정도 확인되지 않아, Semsigrep 경고는 타당합니다.~~

공격 시나리오: 공격자가 nginx 컨테이너 내부에서 원격 코드 실행(RCE) 또는 명령 실행 권한을 획득한 경우, allowPrivilegeEscalation이 제한되지 않아 setuid/setgid 바이너리나 런타임 취약점을 이용해 더 높은 권한으로 상승할 수 있습니다. 이후 컨테이너 내 민감 파일 접근, 다른 프로세스 영향, 마운트된 설정/비밀정보 탈취로 이어질 수 있습니다.

취약 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 10
      periodSeconds: 5
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
    ... (6줄 생략)
```

수정 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities:
        drop: ["ALL"]
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
      initialDelaySeconds: 10
    ... (12줄 생략)
```

수정 가이드: nginx 컨테이너에 securityContext를 추가하고 allowPrivilegeEscalation을 false로 고정합니다. 가능하면 runAsNonRoot, readOnlyRootFilesystem, Linux capability drop까지 함께 적용해 컨테이너 탈취 시 권한 상승과 후속 피해를 최소화합니다. initContainer에도 동일한 정책을 적용하는 것이 권장됩니다.

**ISMS-P:** ISMS-P 2.10.8 패치관리 및 안전한 시스템 운영 취지에 반하며, 컨테이너 보안 설정 미흡으로 인한 침해사고 예방조치 미흡으로 해석될 수 있습니다.

## 10. [MEDIUM] Kubernetes 컨테이너에 권한 상승 방지 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-minikube.yaml`

StatefulSet의 postgres 컨테이너 정의에서 securityContext가 확인되지 않으며, allowPrivilegeEscalation: false가 설정되어 있지 않습니다. Kubernetes 컨테이너는 이미지 내 setuid/setgid 바이너리, 커널 기능, 잘못된 런타임 권한 설정 등에 의해 권한 상승 위험이 발생할 수 있으므로 컨테이너 단위로 권한 상승을 차단해야 합니다. 현재 스니펫 기준으로는 해당 방어 설정이 없어 컨테이너 탈취 후 권한 상승 및 호스트/클러스터 자원 접근 위험이 존재합니다.

공격 시나리오: 공격자가 애플리케이션 취약점(RCE, 파일 업로드 후 코드 실행, 의존성 취약점 등)을 이용해 postgres 컨테이너 내부에서 임의 명령을 실행하면, allowPrivilegeEscalation이 허용된 상태에서 setuid/setgid 바이너리 또는 기타 권한 상승 기법을 시도할 수 있습니다. 이 경우 컨테이너 내부의 낮은 권한에서 더 높은 권한으로 상승해 데이터베이스 파일, 마운트된 볼륨, 서비스 계정 토큰, 클러스터 메타데이터에 접근할 가능성이 커집니다.

취약 코드:

```
spec:
  template:
    spec:
      containers:
        - name: postgres
          image: postgres:15-alpine
          ports:
            - containerPort: 5432
          env:
            - name: POSTGRES_DB
              value: bookdb
            - name: POSTGRES_USER
              valueFrom:
                secretKeyRef:
                  name: db-secret
... (10줄 생략)
```

수정 코드:

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: db
```

```
spec:
  serviceName: "db"
  replicas: 1
  selector:
    matchLabels:
      app: db
  template:
    metadata:
      labels:
        app: db
    spec:
... (29줄 생략)
```

수정 가이드: postgres 컨테이너에 securityContext를 추가하고 allowPrivilegeEscalation을 false로 고정해야 합니다. 가능하면 runAsNonRoot, readOnlyRootFilesystem, capabilities.drop: [ALL]도 함께 적용하여 컨테이너 탈취 시 권한 상승과 추가 피해를 줄이세요. 데이터베이스 이미지 특성상 필요한 경우에만 예외를 검토하고, 예외가 있다면 위험분석과 승인 절차를 문서화해야 합니다.

**ISMS-P: 2.10.2 클라우드 보안 - 클라우드 서비스 이용 시 설정 오류로 중요정보/개인정보 유·노출을 방지하기 위한 보안 설정 기준 및 보호대책 미흡**

## 11. [MEDIUM] Kubernetes DB 워크로드의 기본 보안 설정 미흡

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-minikube.yaml`

제공된 Kubernetes StatefulSet 정의에서 컨테이너 보안 컨텍스트가 확인되지 않아, 기본적으로 권한 상승 제한이 적용되지 않는 구성으로 보입니다. 이는 클라우드 환경에서 비인가 접근 및 설정 오류로 인한 피해를 줄이기 위한 보안 설정 기준 미준수로 볼 수 있습니다.

공격 시나리오: 공격자가 DB 컨테이너 내부 실행권한을 획득하면, 권한 상승 방지가 없어 컨테이너 내부 자격 증명, 마운트된 볼륨 데이터, 프로세스 정보를 더 쉽게 획득할 수 있습니다.

취약 코드:

```
containers:
  - name: postgres
    image: postgres:15-alpine
```

수정 코드:

```
containers:
  - name: postgres
    image: postgres:15-alpine
```

```
securityContext:
  allowPrivilegeEscalation: false
```

수정 가이드: 컨테이너별 securityContext를 명시하여 권한 상승을 차단하세요. 운영 환경에서는 최소권한 원칙에 따라 추가적으로 capabilities 제한과 비루트 실행을 적용하는 것이 바람직합니다.

**ISMS-P: 2.10.2 클라우드 보안**

## 12. [MEDIUM] Kubernetes 컨테이너 권한 상승 방지 설정 미흡

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | `mini_project5\k8s-minikube.yaml`

backend Deployment의 컨테이너에 securityContext가 선언되어 있지 않아 allowPrivilegeEscalation이 명시적으로 false로 제한되지 않습니다. Kubernetes 기본 보안 하드닝이 적용되지 않으면 이미지 내부의 setuid/setgid 바이너리 또는 예상치 못한 실행 경로를 통해 컨테이너 내 권한 상승 가능성이 남습니다.

공격 시나리오: 공격자가 backend 컨테이너 내부에서 코드 실행 권한을 획득했을 때, 이미지 내 setuid/setgid 바이너리나 취약한 실행 파일을 이용해 더 높은 권한으로 상승을 시도할 수 있습니다. 기본 하드닝이 없으면 권한 상승 성공 시 마운트된 볼륨, 환경변수, 프로세스 정보, 인접 자원에 대한 접근 범위가 확대될 수 있습니다.

취약 코드:

```
152:         - name: backend
153:             image: backend:latest
154:             imagePullPolicy: Never
155:             ports:
156:               - containerPort: 8080
157:             resources:
158:               requests:
159:                 cpu: "100m"
160:                 memory: "256Mi"
161:               limits:
162:                 cpu: "500m"
163:                 memory: "512Mi"
```

수정 코드:

```
containers:
  - name: backend
    image: backend:latest
    imagePullPolicy: Never
    securityContext:
      allowPrivilegeEscalation: false
```

```

    runAsNonRoot: true
    capabilities:
      drop: ["ALL"]
    seccompProfile:
      type: RuntimeDefault
  ports:
    - containerPort: 8080
  resources:
    requests:
... (5줄 생략)

```

수정 가이드: backend 컨테이너에 securityContext를 추가하여 권한 상승을 명시적으로 차단해야 합니다. 최소한 allowPrivilegeEscalation을 false로 설정하고, 가능하면 runAsNonRoot, capabilities.drop, seccompProfile(RuntimeDefault)까지 함께 적용하여 컨테이너 하드닝을 강화하십시오.

**ISMS-P:** ISMS-P의 접근통제 및 시스템 보안 설정 미흡에 해당할 수 있습니다. 특히 서비스/시스템 접근 권한을 최소화하고, 원격 또는 실행 환경에서 과도한 권한을 제한해야 한다는 관리적·기술적 보호조치 관점의 미준수로 해석될 수 있습니다.

### 13. [MEDIUM] Kubernetes 컨테이너에 **privilege escalation** 차단 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext | mini_project5\k8s-minikube.yaml`

Kubernetes Deployment의 frontend 컨테이너 정의에 securityContext가 없으며 allowPrivilegeEscalation=false가 설정되어 있지 않습니다. 컨테이너 이미지에 setuid/setgid 바이너리나 추가 권한 획득 경로가 존재할 경우, 컨테이너 내부 프로세스가 권한 상승을 시도할 수 있어 보안상 위험합니다. Semgrep 경고는 설정 누락 자체를 탐지한 것으로, 현재 제공된 스니펫만으로는 이를 무력화하는 방어 로직이 확인되지 않으므로 실제 취약점으로 판단합니다.

공격 시나리오: 공격자가 해당 frontend 컨테이너 내부에서 RCE 또는 취약한 앱 기능을 통해 셸 실행 권한을 획득한 뒤, setuid/setgid 바이너리나 잘못된 권한 설정을 악용해 권한 상승을 시도할 수 있다. 결과적으로 컨테이너 내부에서 더 높은 권한으로 파일/자격증명/마운트 자원에 접근하거나, 호스트 탈출 시도의 발판을 마련할 수 있다.

취약 코드:

```

spec:
  template:
    spec:
      containers:
        - name: frontend
          image: frontend:latest
          imagePullPolicy: ifNotPresent

```

```

    ports:
      - containerPort: 3000
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
        cpu: "500m"
... (1줄 생략)

```

수정 코드:

```

spec:
  template:
    spec:
      containers:
        - name: frontend
          image: frontend:latest
          imagePullPolicy: ifNotPresent
          securityContext:
            allowPrivilegeEscalation: false
            runAsNonRoot: true
            readOnlyRootFilesystem: true
            capabilities:
              drop:
                - ALL
          ports:
... (8줄 생략)

```

수정 가이드: 컨테이너별 securityContext를 추가하여 권한 상승을 차단해야 합니다. 최소한 allowPrivilegeEscalation=false를 적용하고, 가능하면 runAsNonRoot, readOnlyRootFilesystem, capabilities.drop: ["ALL"]까지 함께 설정하여 컨테이너 런타임 권한을 축소하십시오. 이미지가 root로 실행되도록 설계되어 있다면 애플리케이션/도커파일도 함께 수정해야 합니다.

**ISMS-P:** ISMS-P 2.6.1 접근통제 및 안전조치 취지 위반 가능. 컨테이너 권한 통제 미흡은 비인가 권한 상승을 허용할 수 있으며, 내부 관리계획의 접근통제/권한관리/취약점 점검 요구사항과도 연관됩니다.

## 14. [MEDIUM] Kubernetes nginx 컨테이너에 **privilege escalation** 차단 설정 누락

MEDIUM | TP | `yaml.kubernetes.security.allow-privilege-escalation-no-securitycontext.allow-privilege-escalation-no-securitycontext` | `mini_project5\k8s-minikube.yaml`

k8s-minikube.yaml의 nginx Deployment에서 컨테이너 정의는 존재하지만, 해당 컨테이너에 `securityContext.allowPrivilegeEscalation=false` 설정이 보이지 않습니다. 이 경

~~우 이미지 내부에 setuid/setgid 바이너리 또는 예상치 못한 실행 경로가 존재할 때 컨테이너 내 프로세스가 권한 상승을 시도할 수 있어, Pod 보안 경계가 약화됩니다.~~  
Semgrep 규칙이 지정한 내용은 구성상 타당하며, YAML 상으로는 방어 설정이 확인되지 않아 실제 취약점으로 판단합니다.

공격 시나리오: 공격자가 nginx 컨테이너 내부에서 RCE 또는 파일 업로드/템플릿 인젝션 등으로 셸 실행 권한을 확보한 뒤, allowPrivilegeEscalation이 제한되지 않은 상태를 악용해 setuid 바이너리나 권한 상승 가능한 실행 파일을 호출한다. 그 결과 컨테이너 내 root 권한을 획득하거나, 마운트된 설정/볼륨을 통해 민감한 정보에 접근하거나, 추가적인 내부 이동을 시도할 수 있다.

취약 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
    resources:
      requests:
        cpu: "100m"
        memory: "256Mi"
      limits:
        cpu: "500m"
        memory: "512Mi"
... (4줄 생략)
```

수정 코드:

```
containers:
  - name: nginx
    image: nginx:alpine
    securityContext:
      allowPrivilegeEscalation: false
      runAsNonRoot: true
      readOnlyRootFilesystem: true
      capabilities:
        drop:
          - ALL
    ports:
      - containerPort: 80
    readinessProbe:
      tcpSocket:
        port: 80
... (13줄 생략)
```

수정 가이드: 해당 컨테이너에 securityContext를 추가해 privilege escalation을 차단해

아 합니다. 최소한 `allowPrivilegeEscalation=false`를 적용하고, 가능하면 `runAsNonRoot`, `readOnlyRootFilesystem`, `capabilities.drop: ["ALL"]`까지 함께 설정하여 컨테이너 탈취 시 피해 범위를 줄이세요. 또한 배포 전후로 Kubernetes 보안 정책 (PSA/OPA/Gatekeeper 등)으로 동일 설정이 유지되도록 강제하는 것이 좋습니다.

**ISMS-P:** ISMS-P 2.10.2 클라우드 보안, 2.6.1 네트워크 접근통제 관련 보호대책 미흡 가능성. 클라우드 서비스 이용 시 비인가가 접근 및 설정 오류 방지를 위한 보안 설정 기준과 최소권한 원칙을 충족하지 못할 수 있음.

## 15. [LOW] 하드코딩된 샘플 JWT 문자열 탐지

LOW | FP | `generic.secrets.security.detected-jwt-token.detected-jwt-token | mini_project5\book\frontend\src\app\userpage\view\page.js`

클라이언트 코드에 JWT 형식 문자열이 상수로 존재하지만, 주석상 예시값이며 실제 인증 토큰으로 대체 예정입니다. 현재 제공된 문맥에서는 토큰이 전송·검증·권한 부여에 사용되는 흐름이 확인되지 않아 취약점으로 확정할 수 없습니다.

## 16. [INFO] Kubernetes 컨테이너가 root 사용자로 실행될 가능성

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-deployment.yaml`

StatefulSet의 postgres 컨테이너에 `securityContext.runAsNonRoot: true` 설정이 없어, 컨테이너가 root 권한으로 실행될 수 있습니다. 이는 컨테이너 내부 취약점이나 침해 발생 시 권한 상승, 볼륨 내 데이터 접근/변조, 호스트 자원에 대한 영향 확대 위험을 증가시킵니다. Kubernetes 보안 권고상 컨테이너는 가능한 비루트 사용자로 실행되어야 합니다.

## 17. [INFO] Kubernetes 컨테이너 비루트 실행 설정 누락 가능성

INFO | FP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-deployment.yaml`

Kubernetes Pod/Container에 `runAsNonRoot: true`가 없으면 컨테이너가 root로 실행될 수 있어 권한상승 및 컨테이너 탈출 시 피해가 커질 수 있습니다. 다만 본 건은 현재 제공된 코드 조각만으로 해당 설정의 실제 누락 여부를 검증할 수 없어, 확정 취약점으로 판단하지 않았습니다.

## 18. [INFO] Kubernetes 컨테이너의 비루트 실행 보안설정 미흡

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-deployment.yaml`

해당 Kubernetes Deployment의 backend Pod spec에 컨테이너 실행 계정 제한을 명시하는 `securityContext.runAsNonRoot` 설정이 확인되지 않았습니다. 컨테이너가 root

~~권한으로 커딩되거나, 이미지 기본 USER 설정이 root인 경우 권한 상승 및 호스트/마운트 자원 접근 위험이 커집니다. 본 탐지 규칙은 정보성(INFO) 경고이지만, 운영 환경에서는 비루트 실행 강제를 통해 공격 표면을 축소해야 합니다.~~

## 19. [INFO] Kubernetes 컨테이너가 비루트(non-root)로 실행되도록 보안 설정이 필요함

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-deployment.yaml`

제공된 Kubernetes Deployment 스니펫에는 Pod 또는 컨테이너 수준의 securityContext가 보이지 않으며, runAsNonRoot: true 설정도 확인되지 않았습니다. 따라서 nginx 및 initContainer가 root 권한으로 실행될 가능성을 배제할 수 없고, 컨테이너 침해 시 호스트/마운트 자원 접근 범위가 커질 수 있습니다. 이는 ISMS-P 관점에서 안전한 실행권한 최소화 및 취약점 점검·조치 요구사항에 부합하는 미흡 사항입니다.

## 20. [INFO] Kubernetes StatefulSet의 PostgreSQL 컨테이너가 비루트 실행을 보장하지 않음

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-minikube.yaml`

StatefulSet의 postgres 컨테이너에 runAsNonRoot 설정이 확인되지 않아 컨테이너가 root로 실행될 가능성을 배제할 수 없습니다. Kubernetes에서 컨테이너가 root로 실행되면 애플리케이션 취약점 발생 시 권한 상승, 민감 자원 접근, 호스트/볼륨 피해 확대 등의 영향이 커집니다. 본 건은 사용자 입력 기반의 전통적 취약점이 아니라 컨테이너 보안 설정 누락에 따른 구성 취약점입니다.

## 21. [INFO] 컨테이너 실행 권한 최소화 설정 미흡

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-minikube.yaml`

해당 Kubernetes 매니페스트는 postgres 컨테이너에 대한 최소권한 실행 설정을 명시하지 않아, 기본 root 실행 또는 이미지/런타임 의존 실행이 발생할 수 있습니다. 이는 운영 환경에서의 권한 분리 원칙에 위배되며, 침해 발생 시 피해 범위를 확대시킬 수 있습니다.

## 22. [INFO] Kubernetes 컨테이너 비루트 실행 설정 누락

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root | mini_project5\k8s-minikube.yaml`

frontend Deployment의 컨테이너 spec에 runAsNonRoot 또는 runAsUser를 지정한 securityContext가 보이지 않습니다. 따라서 컨테이너가 root로 실행될 가능성을 배제

할 수 없고, 이미지 내부 설정에 의존하게 됩니다. Kubernetes 환경에서 root 실행은 컨테이너 탈출, 권한 상승, 호스트 자원 접근 범위를 넓힐 수 있으므로 비루트 실행을 명시적으로 강제하는 것이 안전합니다.

## 23. [INFO] Kubernetes Pod/Container가 비루트 사용자 실행 강제 설정 없이 배포됨

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` | `mini_project5\k8s-minikube.yaml`

제공된 스니펫에는 Deployment의 Pod template spec 하에 `securityContext.runAsNonRoot: true` 또는 컨테이너별 `securityContext`가 설정되어 있지 않습니다. nginx 본 컨테이너와 busybox initContainer가 기본값으로 root로 실행될 수 있어, 컨테이너가 탈취될 경우 권한 상승 및 호스트/클러스터 내 민감 자원 접근 위험이 증가합니다. 다만 본 분석에서는 전체 파일 원문을 직접 재확인하지 못했으므로, 완전한 경로 증명은 스니펫 기준으로만 판단했습니다.

## 24. [INFO] Kubernetes 컨테이너가 비루트 실행 보장이 없어 권한 상승 위험이 있음

INFO | TP | `yaml.kubernetes.security.run-as-non-root.run-as-non-root` | `mini_project5\k8s-minikube.yaml`

Deployment의 backend Pod spec에서 container securityContext에 `runAsNonRoot` 설정이 확인되지 않았습니. 이 경우 이미지/엔트리포인트 기본값에 따라 컨테이너가 root로 실행될 수 있으며, 애플리케이션 취약점이 발생했을 때 권한 상승 및 호스트/클러스터 자원 접근 위험이 커집니다. 제공된 스니펫 기준으로는 명시적 방어 설정이 없어 Semgrep 탐지는 실제 구성 취약점(True Positive)으로 판단합니다.

## 25. [INFO] initContainer/컨테이너 비루트 실행 설정 미흡

INFO | FP | N/A | `mini_project5\k8s-minikube.yaml`

initContainer와 backend 컨테이너에 대해 비루트 실행을 보장하는 securityContext가 보이지 않아, 컨테이너 기본 실행 계정에 따라 root 권한으로 동작할 수 있습니다. 이는 직접적인 코드 취약점이라기보다 쿠버네티스 배포 구성의 하드닝 미흡입니다.

## 2.2 비즈니스 로직 심층 분석 결과

### 1. [HIGH] 클라이언트가 `userId`를 직접 지정하는 작품 등록 API로 인한 IDOR 가능성

HIGH | IDOR | `mini_project5\book\frontend\src\app\userpage\create\page.js`

~~프론트엔드가 작품 등록 요청 시 /api/book/insertByUrl?~~

userId=\${userData.userId} 형태로 사용자 식별자를 쿼리스트링에 실어 보냅니다. 이 값은 로컬스토리지 기반이라 사용자가 임의로 변경 가능하며, 백엔드가 세션/토큰의 실제 인증 주체와 대조하지 않으면 다른 사용자의 userId로 작품을 등록하는 권한 우회(IDOR)가 발생할 수 있습니다.

공격 시나리오: 공격자가 브라우저 개발자도구로 localStorage의 user 객체 또는 요청 URL의 userId를 자신의 값에서 타인의 값으로 바꿉니다. 이후 작품 등록을 실행하면 서버가 쿼리 파라미터를 신뢰하는 경우, 공격자 데이터가 타 사용자 계정으로 생성되거나 타 사용자 소유 데이터가 오염됩니다.

취약 코드:

```
const userData = JSON.parse(localStorage.getItem("user"));
...
const response = await fetch(`/api/book/insertByUrl?userId=${userData.userId}`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify(bookData),
});
```

수정 코드:

```
const response = await fetch(`/api/book/insertByUrl`, {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": `Bearer ${accessToken}`
  },
  body: JSON.stringify(bookData),
});
```

수정 가이드: userId를 클라이언트가 전달하지 않도록 변경하고, 백엔드에서 인증 토큰/세션에서 식별한 사용자로만 소유자를 결정해야 합니다. 부득이하게 식별자를 전달해야 한다면 서버에서 토큰의 주체와 일치 여부를 검증하고 불일치 시 403으로 차단해야 합니다.

**ISMS-P:** ISMS-P 3. 개인정보 보호 조직/접근 권한 관리, 6. 접근 통제에 관한 사항 위반 가능성

## 2. [HIGH] 비밀번호 변경 API에 클라이언트 저장 userId를 그대로 사용하여 계정 바인딩 우회 가능

HIGH | IDOR |

mini\_project5\book\frontend\src\app\password\change\page.js

현재 비밀번호 변경 화면은 로그인 식별자를 서버 세션/JWT에서 재확인하지 않고,

~~localStorage의 user 객체에서 읽은 userId를 그대로 요청 본문에 포함합니다. 이 구조는 서버측 /api/user/change-pw가 요청 body의 userId를 신뢰하는 경우, 공격자가 브라우저 저장값을 조작해 다른 계정의 비밀번호 변경을 시도할 수 있는 IDOR/권한회피로 이어질 수 있습니다. 클라이언트 코드만으로는 서버 검증 여부를 확인할 수 없지만, 제공된 흐름상 안전한 사용자 바인딩이 보장되지 않습니다.~~

공격 시나리오: 공격자가 자신의 브라우저 localStorage.user 값을 수정해 다른 피해자의 userId로 바꾼 뒤 비밀번호 변경 요청을 전송합니다. 서버가 세션 주체와 body.userId를 대조하지 않으면 피해자 계정의 현재 비밀번호 검증/변경이 수행되어 계정 탈취로 이어질 수 있습니다.

취약 코드:

```
const user = JSON.parse(stored);
setUserId(user.userId);
...
body: JSON.stringify({
  userId,
  currentPassword,
  newPassword,
}),
```

수정 코드:

```
const res = await fetch('/api/user/change-pw', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  credentials: 'include',
  body: JSON.stringify({
    currentPassword,
    newPassword,
  }),
});
```

수정 가이드: 사용자 식별자는 클라이언트 저장소 값이 아니라 서버 세션, HttpOnly 쿠키, 또는 검증된 JWT의 subject에서 결정해야 합니다. 비밀번호 변경 API는 요청 본문의 userId를 받지 말고, 서버가 인증 컨텍스트에서 대상 계정을 식별하도록 수정해야 합니다. 또한 현재 비밀번호 검증은 반드시 서버측에서 수행하고, 요청자와 대상 계정 일치 여부를 강제해야 합니다.

**ISMS-P:** 개인정보 보호법 제29조 안전조치의무, ISMS-P 2.6 접근통제 위반 가능성

### 3. [HIGH] 비밀번호 찾기 결과로 임시 비밀번호를 직접 노출하는 계정 복구 로직

HIGH | Logic Bypass | mini\_project5/book/frontend/src/app/find/password/page.js

~~프론트엔드가 /api/user/find-pw 응답의 result.data.tempPassword를 그대로 화면에 출력합니다.~~ 계정 복구 과정에서 임시 비밀번호를 직접 반환하는 방식은 이메일/이름/전화번호 조합만 맞으면 공격자가 비밀번호를 획득할 수 있는 구조로 이어질 수 있습니다. 백엔드에 추가 인증, 일회성 토큰, 강한 레이트리미트가 없으면 계정 탈취로 바로 연결되는 실질적 비즈니스 로직 결함입니다.

공격 시나리오: 공격자가 대상 사용자의 이메일, 이름, 전화번호를 추정/수집한 뒤 /api/user/find-pw를 반복 호출합니다. 서버가 추가 검증 없이 tempPassword를 내려주면 해당 임시 비밀번호를 획득해 즉시 로그인할 수 있고, 이후 비밀번호 변경 전까지 계정을 장악할 수 있습니다.

취약 코드:

```
const result = await response.json();

if (!response.ok || result.status !== "success") {
  alert(result.message || "비밀번호 찾기에 실패했습니다.");
  return;
}

// 임시 비밀번호 화면에 보여주기
setTempPassword(result.data.tempPassword);
```

수정 코드:

```
const result = await response.json();

if (!response.ok || result.status !== "success") {
  alert(result.message || "비밀번호 찾기에 실패했습니다.");
  return;
}

alert("인증된 연락처로 비밀번호 재설정 링크를 전송했습니다.");
router.push('/login');
```

수정 가이드: 임시 비밀번호를 응답 본문이나 화면에 직접 노출하지 말고, 본인 확인 후 이메일/SMS로 일회성 재설정 링크를 발급하는 방식으로 변경해야 합니다. 재설정 토큰은 만료시간과 1회 사용 제한을 두고, IP/계정별 레이트리미트와 실패 횟수 제한을 서버에서 강제해야 합니다.

**ISMS-P:** ISMS-P 1.1.6, 2.8.6

## 4. [HIGH] API 키 검증과 사용자 인증이 없는 이미지 생성 프록시 엔드포인트

HIGH | Logic Bypass |

mini\_project5\book\frontend\src\app\api\generateCover\route.js

이 페이지는 작품 목록 조회 시 localStorage의 user.userId를 그대로 /api/book/list/my?userId=...로 전송하고, 삭제/수정도 각각 작품 ID만으로 호출합니다. 즉, 서버가 세션/토큰의 실제 사용자와 리소스 소유권을 강제 검증하지 않으면 공격자가 localStorage나 요청 파라미터를 조작해 타인의 작품을 조회·수정·삭제하는 IDOR가 가능합니다. 프론트에서 소유권을 신뢰하는 구조이므로 백엔드 검증이 없을 경우 실제 exploit이 가능한 권한 우회입니다.

공격 시나리오: 공격자는 브라우저 DevTools에서 localStorage의 userId를 다른 계정 값으로 바꾸거나, 네트워크 요청을 프록시에서 수정해 /api/book/list/my?userId=피해자ID를 호출하여 타인의 작품 목록을 확인합니다. 이후 목록에서 얻은 bookId를 이용해 /api/book/delete/{bookId} 또는 /api/book/update/{bookId}를 직접 호출하면, 서버가 소유자 검증을 하지 않는 경우 피해자 작품을 삭제/수정할 수 있습니다.

취약 코드:

```
const user = typeof window !== "undefined" ? localStorage.getItem("user")
const parsed = JSON.parse(user);
const userId = parsed.userId;
const response = await fetch(`/api/book/list/my?userId=${userId}`, { ... })
...
await fetch(`/api/book/delete/${idToDelete}`, { method: "DELETE" });
...
formData.append("userId", user.userId);
await fetch(`/api/book/update/${editingWork.id}`, { method: "PUT", body: f
```

수정 코드:

```
const response = await fetch('/api/book/list/my', {
  method: 'GET',
  headers: {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${accessToken}`
  }
});

await fetch(`/api/book/delete/${idToDelete}`, {
  method: 'DELETE',
  headers: { 'Authorization': `Bearer ${accessToken}` }
});

const formData = new FormData();
formData.append('book', new Blob([bookJson], { type: 'application/json' }));
... (5줄 생략)
```

수정 가이드: 리소스 소유권 판단을 클라이언트의 userId나 path parameter에 의존하지 말고, 서버에서 인증 토큰/세션의 사용자 식별자와 book owner를 강제 매칭해야 합니다. 목록 조회는 userId 쿼리 제거 후 서버가 현재 로그인 사용자 기준으로 필터링하고, 수정/삭제는 요청된 bookId가 현재 사용자 소유인지 서버에서 검증한 뒤 처리해야 합니다.

**ISMS-P:** 개인정보보호법 제29조(안전조치의무) 및 ISMS-P 접근통제/권한관리 취지 위반 가능. 타인의 개인정보성 콘텐츠(작품 정보)에 대한 비인가 조회·수정·삭제를 방지하는 서버측 접근통제 미흡에 해당할 수 있음.

## 6. [HIGH] 비밀번호 변경 API에서 세션 사용자 검증 없이 요청 `userId`를 신뢰하는 권한 우회(IDOR)

HIGH | IDOR |

mini\_project5\book\src\main\java\com\aiivle0102\book\controller\AuthControlle

`/user/change-pw`는 요청 바디의 `userId`를 그대로

`userService.changePassword()`에 전달하며, 현재 세션의 로그인 사용자와 동일한지 확인하지 않습니다. 따라서 공격자는 본인 계정이 아닌 다른 계정의 `userId`를 넣어 비밀번호 변경을 시도할 수 있습니다. 실제 성공 여부는 서비스 내부 검증에 좌우되지만, 컨트롤러 레벨에서 객체 소유권 검증이 없으므로 비즈니스 로직상 명백한 권한 우회 위험이 존재합니다.

공격 시나리오: 공격자가 자신의 세션으로 로그인한 뒤, 요청 본문에 피해자의 `userId`를 넣고 현재 비밀번호를 알아내거나 서비스 내부 검증이 느슨한 경우 비밀번호 변경을 시도합니다. 계정 식별자가 순차적이거나 유추 가능하면 공격 난이도가 더 낮아집니다.

취약 코드:

```
@PostMapping("/change-pw")
public ResponseEntity<?> changePassword(
    @Valid @RequestBody ChangePasswordRequest request,
    BindingResult bindingResult
) {
    ...
    userService.changePassword(
        request.getUserId(),
        request.getCurrentPassword(),
        request.getNewPassword()
    );
    ...
}
```

수정 코드:

```
@PostMapping("/change-pw")
public ResponseEntity<?> changePassword(
    @Valid @RequestBody ChangePasswordRequest request,
    BindingResult bindingResult,
    HttpSession session
) {
    if (bindingResult.hasErrors()) {
        return ResponseEntity.status(HttpStatus.BAD_REQUEST)
            .body(Map.of("status", "error", "message", bindingResult.g
```

```

    }

    Long sessionUserId = (Long) session.getAttribute("user");
    if (sessionUserId == null || !sessionUserId.equals(request.getUserId())
        return ResponseEntity.status(HttpStatus.FORBIDDEN)
            .body(Map.of("status", "error", "message", "본인 계정만 변경할 수 있습니다."));
    ... (5줄 생략)

```

수정 가이드: 비밀번호 변경은 반드시 세션의 인증 주체와 요청 대상 계정이 일치할 때만 허용해야 합니다. 가능하면 `userId`를 요청 바디에서 받지 말고 세션/토큰에서 식별자를 직접 추출해 서비스에 전달하도록 변경하세요. 또한 서비스 계층에서도 최종 소유권 검증을 수행해 이중 방어를 적용해야 합니다.

**ISMS-P:** 정보주체의 계정 정보에 대한 접근통제 미흡, 권한 최소화 및 계정별 접근통제 정책 위반 가능

## 7. [HIGH] 본인 확인 절차 없이 임시 비밀번호를 발급하는 비밀번호 찾기 흐름

HIGH | Logic Bypass |

mini\_project5\book\src\main\java\com\aiivle0102\book\controller\AuthControlle

`/user/find-pw`는 email/name/phone 조합만으로 `userService.resetPassword()`를 호출하고, 발급된 임시 비밀번호를 응답 본문에 그대로 반환합니다. 세션 인증, 추가 인증(OTP/메일 링크), 레이트리밋, 시도 제한이 컨트롤러에서 보이지 않아 타인의 개인정보 일부만 알아도 계정 탈취로 이어질 수 있습니다. 비밀번호 재설정이 사실상 무권한으로 자동 처리되므로 비즈니스 로직 결함으로 판단됩니다.

공격 시나리오: 공격자가 유출되었거나 추정 가능한 이메일, 이름, 전화번호를 조합해 `/find-pw`를 반복 호출하면 임시 비밀번호를 직접 획득할 수 있습니다. 이후 해당 임시 비밀번호로 로그인하여 계정을 장악할 수 있습니다.

취약 코드:

```

@PostMapping("/find-pw")
public ResponseEntity<?> findPassword(@RequestBody Map<String, String> req
    String email = request.get("email");
    String name = request.get("name");
    String phone = request.get("phone");

    String tempPassword = userService.resetPassword(email, name, phone);
    ...
    data.put("tempPassword", tempPassword);
    return ResponseEntity.ok(body);
}

```

수정 코드:

```

@Override
public UserInfo getUser(UserInfo user) {
    String email = user.getEmail();
    String pw = user.getPassword();

    UserInfo found = userInfoRepository.findByEmail(email)
        .orElseThrow(() -> new RuntimeException("로그인 실패"));

    if (!passwordEncoder.matches(pw, found.getPassword())) {
        throw new RuntimeException("로그인 실패");
    }
    return found;
}

```

수정 가이드: 비밀번호를 DB 조회 조건에 포함하지 말고, 이메일로 사용자만 조회한 뒤 BCrypt 같은 검증기를 사용해 입력 비밀번호와 저장된 해시를 비교해야 합니다. 저장 시에도 반드시 해시를 적용하고, 인증 실패 메시지는 동일하게 유지해 계정 존재 여부가 노출되지 않도록 합니다.

**ISMS-P:** 2.5.5 특수 계정 및 권한 관리, 2.9.1 변경관리, 개인정보의 안전성 확보조치 기준의 인증정보 보호 취지 위반 가능

## 9. [HIGH] 사용자 비밀번호 변경 API의 권한 검증 부재로 인한 IDOR

HIGH | IDOR |

mini\_project5\book\src\main\java\com\aiivle0102\book\service\impl\UserService

changePassword(Long userId, ...)는 호출자가 해당 userId의 소유자인지 확인하지 않고, 전달된 ID만으로 사용자를 조회한 뒤 비밀번호를 변경합니다. 인증된 공격자가 다른 사용자의 userId만 알면 타 계정의 비밀번호를 임의로 변경할 수 있어 계정 탈취로 이어질 수 있습니다. 서비스 계층에 현재 사용자와 대상 사용자 간의 일치 검증 또는 관리자 권한 검증이 없으므로 실제 악용 가능한 Broken Access Control 결함입니다.

공격 시나리오: 공격자가 자신의 세션으로 비밀번호 변경 기능을 호출하면서 요청의 userId를 다른 사용자의 값으로 바꾸고, 현재 비밀번호는 해당 계정의 실제 값이 아니므로 일반적으로 막히지만, 만약 현재 비밀번호 검증 로직이 별도 컨트롤러/호름에서 우회되거나 유출된 정보가 있으면 타 계정 비밀번호를 직접 변경할 수 있습니다. 무엇보다 서비스 로직 자체에 소유권 검증이 없어 상위 계층 실수 시 즉시 IDOR가 됩니다.

취약 코드:

```

public void changePassword(Long userId, String currentPassword, String newPassword) {

    UserInfo user = userInfoRepository.findById(userId)
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다"));

    if (!user.getPassword().equals(currentPassword)) {

```

```

        throw new IllegalArgumentException("현재 비밀번호가 일치하지 않습니다.");
    }

    if (currentPassword.equals(newPassword)) {
        throw new IllegalArgumentException("새 비밀번호는 이전 비밀번호와 같을 수 없습니다.");
    }

    user.setPassword(newPassword);
    userInfoRepository.save(user);
    ... (1줄 생략)

```

수정 코드:

```

public void changePassword(Long targetUserId, Long currentUserId, String newPassword) {
    if (!targetUserId.equals(currentUserId)) {
        throw new AccessDeniedException("본인 계정만 변경할 수 있습니다.");
    }
    UserInfo user = userInfoRepository.findById(targetUserId)
        .orElseThrow(() -> new IllegalArgumentException("사용자를 찾을 수 없습니다."));
    if (!passwordEncoder.matches(currentPassword, user.getPassword())) {
        throw new IllegalArgumentException("현재 비밀번호가 일치하지 않습니다.");
    }
    if (passwordEncoder.matches(newPassword, user.getPassword())) {
        throw new IllegalArgumentException("새 비밀번호는 이전 비밀번호와 달라야 합니다.");
    }
    user.setPassword(passwordEncoder.encode(newPassword));
    userInfoRepository.save(user);
}

```

수정 가이드: 비밀번호 변경 시 대상 계정과 요청 주체의 소유권을 반드시 검증하고, 관리자 전용 기능이 아니라면 본인 계정만 변경 가능하도록 제한해야 합니다. 또한 평문 비교 대신 안전한 비밀번호 검증/해시 저장을 적용해야 합니다.

**ISMS-P: 2.6.1 접근통제:** 인가되지 않은 계정 식별자(userId)를 이용한 자원 접근/변경을 통제하지 않음

## 10. [MEDIUM] 로그인 성공 후 사용자 식별 정보를 클라이언트 `localStorage`에 저장

MEDIUM | Broken Access Control |  
mini\_project5\book\frontend\src\app\login\page.js

로그인 성공 시 서버가 반환한 `userId`와 `email`을 브라우저 `localStorage`의 'user' 키에 저장합니다. 이후 애플리케이션의 다른 화면이나 API가 이 값을 인증/권한 판단에 사용하면, 사용자가 브라우저 콘솔로 값을 임의 조작하여 다른 계정으로 위장하거나 IDOR를 유발할 수 있습니다. 이 파일 단독으로는 후속 사용처가 없지만, 실제 브라우저 저장소 기반 인증 패턴은 서버 신뢰가 아닌 클라이언트 신뢰로 이어져 접근통제 결함의 전형적인 전단계가 됩니다.

공격 시나리오: 공격자가 로그인 후 DevTools에서 localStorage.user 값을 임의의 userId/email로 변경합니다. 만약 메인 페이지나 다른 API가 이 값을 기준으로 사용자 정보를 조회하거나 권한을 결정하면, 타 사용자 데이터 열람/수정 또는 계정 위장이 가능해집니다.

취약 코드:

```
localStorage.setItem("user", JSON.stringify({
  userId: result.userId,
  email: result.email
}));
```

수정 코드:

```
// 클라이언트 저장소에 권한 판단용 식별자를 저장하지 않는다.
// 서버 세션/HttpOnly Secure SameSite 쿠키 기반으로 인증 상태를 관리한다.
// 예시: 서버가 세션 쿠키를 발급하고 프론트는 /api/me로만 사용자 정보를 조회
const response = await fetch('/api/user/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ email, password }),
  credentials: 'include'
});

if (!response.ok) {
  alert('로그인 실패');
  return;
}

... (1줄 생략)
```

수정 가이드: 사용자 식별자와 인증 상태를 localStorage에 저장하지 말고, 서버가 관리하는 세션 또는 HttpOnly 쿠키로 처리해야 합니다. 프론트는 단순히 로그인 결과만 받고, 사용자 정보가 필요하면 인증된 백엔드 API를 통해 조회하도록 변경하세요.

**ISMS-P: 2.5.5** 특수 계정 및 권한 관리 / 1.2.1 정보자산 식별

## 11. [MEDIUM] 로그인 성공 조건을 응답 본문의 **status** 값에 의존

MEDIUM | Logic Bypass |

mini\_project5\book\frontend\src\app\login\page.js

로그인 성공 여부를 HTTP 상태코드만이 아니라 result.status === 'success'에 의존해 판정합니다. 서버가 에러를 잘못된 형태로 내려주거나 status 필드가 누락/예측 가능하게 처리되면, 클라이언트가 성공 흐름으로 진입할 여지가 생깁니다. 이 자체만으로 즉시 우회가 확정되는 것은 아니지만, 인증 로직이 느슨한 응답 계약에 묶여 있어 비즈니스 로직 결함으로 이어질 수 있습니다.

공격 시나리오: 서버가 예외 상황에서 HTTP 200과 함께 status 필드를 부정확하게 반환하

거나, 중간 로직 오류로 success가 내려오는 경우 사용자는 실제 인증 실패 상태인데도 로그인 성공 처리됩니다. 이후 후속 화면이 클라이언트 저장값을 신뢰하면 계정 상태 불일치가 발생합니다.

취약 코드:

```
if (!response.ok || result.status !== "success") {
    alert("✖ 로그인 실패! 이메일 또는 비밀번호를 확인하세요.");
    return;
}

localStorage.setItem("user", JSON.stringify({
    userId: result.userId,
    email: result.email
}));

alert("✔ 로그인 성공!");
window.location.href = "/mainpage";
```

수정 코드:

```
const response = await fetch('/api/user/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password }),
    credentials: 'include'
});

if (!response.ok) {
    alert('로그인 실패');
    return;
}

// 서버가 세션/쿠키를 발급했는지 별도 확인하거나,
// /api/me 같은 인증된 엔드포인트로 상태를 검증한다.
window.location.href = '/mainpage';
```

수정 가이드: 로그인 성공 여부는 UI용 문자열이 아니라 서버의 인증 결과와 세션 발급 여부로 판단해야 합니다. 응답 본문 필드에만 의존하지 말고, 인증 완료 후 보호된 사용자 조회 API로 현재 로그인 상태를 검증하는 구조로 변경하세요.

ISMS-P: 2.9.1 변경관리

## 12. [MEDIUM] 요청량 제한 부재로 인한 비용 고갈 및 이상 트래픽 통제 미흡

MEDIUM | Logic Bypass |

mini\_project5\book\frontend\src\app\api\generateCover\route.js

~~엔드포인트에 레이트리밋, 쿼터, 호출자별 사용량 제한이 없습니다. prompt 길이 제한도 없어 대량/반복 호출로 외부 API 비용과 서버 자원을 소모시킬 수 있습니다.~~ 이는 단순 성능 이슈가 아니라, 외부 API 과금 유발과 서비스 남용 방지 통제가 없는 비즈니스 로직 결함입니다.

공격 시나리오: 공격자가 스크립트로 이 엔드포인트를 반복 호출하면 OpenAI 이미지 생성 요청이 지속적으로 발생합니다. 서비스 운영자는 예산 초과나 응답 지연을 겪을 수 있고, 정상 사용자의 요청이 실패할 수 있습니다.

취약 코드:

```
const { apiKey, model, prompt } = await req.json();
...
const openaiRes = await fetch("https://api.openai.com/v1/images/generation
```

수정 코드:

```
// 예시: 사용자별 레이트리밋과 입력 제한 추가
if (!checkRateLimit(user.id)) {
  return NextResponse.json({ error: "Too Many Requests" }, { status: 429 })
}
if (!prompt || prompt.length > 1000) {
  return NextResponse.json({ error: "Invalid prompt" }, { status: 400 });
}
```

수정 가이드: 호출자별 레이트리밋, 일일 쿼터, prompt 길이 제한, 실패 횟수 제한을 도입해야 합니다. 또한 이상행위 모니터링과 차단 로직을 추가해 대량 호출을 통제해야 합니다.

**ISMS-P:** 176 이상행위 분석 및 모니터링

### 13. [MEDIUM] 클라이언트 제어 **model** 값에 대한 허용 목록 검증 부재

MEDIUM | Logic Bypass |

mini\_project5\book\frontend\src\app\api\generateCover\route.js

**model** 값을 클라이언트가 임의로 지정할 수 있으며, 서버는 기본값만 제공하고 별도의 allowlist 검증을 하지 않습니다. 서비스 정책상 특정 모델만 허용해야 하는 경우 이 로직은 정책 우회 지점이 됩니다. 잘못된 모델을 넣는 입력 오류 수준을 넘어, 허용되지 않은 비용 구조나 기능을 사용하게 할 수 있습니다.

공격 시나리오: 공격자가 비용이 더 큰 모델이나 정책상 금지된 모델명을 넣어 요청하면, 서버는 이를 그대로 전달합니다. 운영 정책이 '특정 모델만 사용'인 경우 이를 무력화할 수 있습니다.

취약 코드:

```
body: JSON.stringify({
  model: model || "dall-e-3",
  prompt,
  n: 1,
  size: "1024x1024"
})
```

수정 코드:

```
const allowedModels = new Set(["dall-e-3"]);
if (!allowedModels.has(model ?? "dall-e-3")) {
  return NextResponse.json({ error: "Unsupported model" }, { status: 400 })
}
```

수정 가이드: 서버에서 허용 모델을 명시적으로 제한하고, 클라이언트 입력은 검증 후 거부해야 합니다. 정책상 단일 모델만 사용할 경우 아예 서버 고정값으로 처리하는 것이 가장 안전합니다.

**ISMS-P: 2.5.5** 특수 계정 및 권한 관리

## 14. [MEDIUM] 회원가입 이메일 중복 확인과 저장 사이의 경쟁조건

MEDIUM | Race Condition |

mini\_project5\book\src\main\java\com\aivle0102\book\service\impl\UserService

signUp()은 먼저 findByEmail()로 존재 여부를 확인한 뒤, 그 결과가 없으면 save()를 수행하는 체크-후-저장 패턴입니다. 두 개의 동시 요청이 같은 이메일로 들어오면 둘 다 중복이 없다고 판단하고 저장 단계로 진행할 수 있어, DB 유니크 제약이 없을 경우 중복 계정 생성이 가능합니다. 유니크 제약이 있더라도 애플리케이션 로직만으로는 경쟁조건을 막지 못합니다.

공격 시나리오: 공격자가 동일 이메일로 가입 요청을 짧은 시간에 여러 개 동시에 보내면, 각 요청이 중복 미존재로 판단한 뒤 저장되어 동일 이메일 계정이 중복 생성될 수 있습니다. 이로 인해 계정 식별, 인증 흐름, 비밀번호 재설정 대상 식별이 꼬이거나 비정상 계정 생성이 발생할 수 있습니다.

취약 코드:

```
userInfoRepository.findByEmail(request.getEmail())
    .ifPresent(user -> {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    });
...
return userInfoRepository.save(user);
```

수정 코드:

```

@Transactional
public UserInfo signUp(UserSignUpRequest request) {
    String email = request.getEmail().trim().toLowerCase();
    UserInfo user = new UserInfo();
    user.setEmail(email);
    user.setPassword(passwordEncoder.encode(request.getPassword()));
    user.setName(request.getName().trim());
    user.setPhone(request.getPhone() != null ? request.getPhone().trim() :
    try {
        return userInfoRepository.save(user);
    } catch (DataIntegrityViolationException e) {
        throw new IllegalStateException("이미 가입된 이메일입니다.");
    }
}
}

```

수정 가이드: 이메일 컬럼에 DB 유니크 제약을 두고, 애플리케이션에서는 사전 조회에 의존하지 말고 저장 시 충돌을 예외로 처리해야 합니다. 동시 요청에도 일관성이 유지되도록 원자성을 DB 레벨에서 보장하는 것이 핵심입니다.

**ISMS-P: 2.8.6** 운영환경 이관: 운영계에서 동시성/정합성 문제를 유발할 수 있는 변경을 통제된 절차와 검증 없이 반영한 상태

## 15. [INFO] 하드코딩 시크릿은 확인되지 않음

INFO | Hardcoded Secrets |  
mini\_project5\book\frontend\src\app\userpage\create\page.js

코드 상에서 DB 비밀번호, API Key, 토큰 등의 하드코딩 값은 발견되지 않았습니다.

## 16. [INFO] 비밀번호 복구 API의 서버측 통제 여부 미확인

INFO | Logic Bypass | mini\_project5\book\frontend\src\app\find\password\page.js

이 파일은 비밀번호 찾기 요청만 수행하므로, 실제 취약성의 성패는 /api/user/find-pw 백엔드 구현에 달려 있습니다. 서버측에서 추가 본인인증과 호출 제한을 충분히 수행한다면 이 프론트 코드는 곧바로 취약점이 되지 않을 수 있습니다.

## 17. [INFO] 비밀번호 변경 요청에 클라이언트 지정 userId 포함으로 인한 IDOR 가능성 검토 필요

INFO | IDOR |  
mini\_project5\book\src\main\java\com\aiivle0102\book\dto\ChangePasswordRequest

ChangePasswordRequest에 userId가 요청 바디로 노출되어 있어, 상위 계층에서 이를 그대로 신뢰하면 다른 사용자의 계정에 대해 비밀번호 변경이 수행될 수 있다. 다만 이 파일은 단순 입력 DTO이므로 실제 취약점은 인증 컨텍스트와의 비교/검증 여부에

따라 결정된다.

---

## 18. [INFO] 현재 비밀번호 검증 수행 여부는 상위 로직 확인 필요

INFO | Logic Bypass |

mini\_project5\book\src\main\java\com\aivle0102\book\dto\ChangePasswordReques

DTO는 currentPassword와 newPassword를 수신할 뿐이며, 현재 비밀번호가 어떤 계정에 대해 검증되는지 보장하지 않는다. 이 자체로는 취약점 확정이 불가능하고, 서비스 계층에서 인증 사용자와 계정 매칭 여부를 추가 확인해야 한다.

본 리포트는 자동화 소스코드 취약점 점검을 통해 산출된 보안 감사 문서입니다.

본 결과는 참고용이며 배포 전 보안 부서의 추가 검토가 필요할 수 있습니다.